

DLSEM Practical Programs

8 Python Codes

Program 1: ex1.py

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# XOR input and output
X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 1, 1, 0])

# DNN model
model = Sequential([
    Dense(4, activation='relu', input_shape=(2,)),
    Dense(1, activation='sigmoid')
])

# Compile
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Train
model.fit(X, y, epochs=500, verbose=0)

# Prediction
print("Predictions:")
print((model.predict(X) > 0.5).astype(int))
```

Program 2: ex2.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from sklearn.model_selection import train_test_split

# Load static A-Z dataset
data = pd.read_csv("A_Z Handwritten Data.csv")

X = data.iloc[:,1:].values / 255.0
y = data.iloc[:,0].values

X = X.reshape(-1,28,28,1)

X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.2,random_state=42)

# CNN
model = Sequential([
    Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128,activation='relu'),
    Dense(26,activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train,y_train,epochs=5,batch_size=64)

# Predict one test image
i = 10
pred = np.argmax(model.predict(X_test[i:i+1]))

# Display image and prediction
plt.imshow(X_test[i].reshape(28,28),cmap='gray')
plt.title("Actual: " + chr(y_test[i]+65) +
          " | Predicted: " + chr(pred+65))
plt.axis("off")
plt.show()
```

Program 3: ex3.py

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_lfw_people
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Load LFW dataset
data = fetch_lfw_people(min_faces_per_person=70, resize=0.4)

X = data.images / 255.0
y = data.target

X = X.reshape(-1, X.shape[1], X.shape[2], 1)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# CNN
model = Sequential([
    Conv2D(32, (3,3), activation='relu',
          input_shape=X_train.shape[1:]),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(len(data.target_names), activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=10, batch_size=32)

# Accuracy
print("Accuracy:", model.evaluate(X_test, y_test)[1])

# Prediction and image output
i = 5
pred = np.argmax(model.predict(X_test[i:i+1], verbose=0))

plt.imshow(X_test[i].reshape(X_test.shape[1], X_test.shape[2]),
           cmap='gray')
plt.title("Actual: " + data.target_names[y_test[i]] +
         "\nPredicted: " + data.target_names[pred])
plt.axis("off")
plt.show()
```

Program 4: ex4.py

```
"""
import torch
import torch.nn as nn
import glob, string, random

chars = string.ascii_letters + " .,;"
n = len(chars)

# Read names
files = glob.glob("data/names/*.txt")
names = []
for f in files:
    names += [x.strip() for x in open(f, encoding="utf8")]

# RNN
class RNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.RNN(n, 64)
        self.fc = nn.Linear(64, n)

    def forward(self, x, h):
        y, h = self.rnn(x, h)
        return self.fc(y), h

model = RNN()
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters(), lr=0.01)

# Training
for _ in range(5000):
    name = random.choice(names)
    x = torch.zeros(len(name)-1, 1, n)
    y = torch.tensor([chars.index(c) for c in name[1:]])

    for i, c in enumerate(name[:-1]):
        x[i, 0, chars.index(c)] = 1

    out, _ = model(x, torch.zeros(1, 1, 64))
    loss = loss_fn(out.squeeze(1), y)

    opt.zero_grad()
    loss.backward()
    opt.step()

# Generate name
c = random.choice(string.ascii_letters)
result = c

for _ in range(10):
    x = torch.zeros(1, 1, n)
    x[0, 0, chars.index(c)] = 1
    out, _ = model(x, torch.zeros(1, 1, 64))
    c = chars[out.argmax().item()]
    result += c

print("Generated name:", result)"""

import torch
import torch.nn as nn

names = ["anna", "alex", "arun", "john", "james",
        "maria", "mark", "david", "lisa", "robert"]

chars = "abcdefghijklmnopqrstuvwxyz"
n = len(chars)

class RNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.RNN(n, 32)
        self.fc = nn.Linear(32, n)

    def forward(self, x):
        y, _ = self.rnn(x)
        return self.fc(y)

model = RNN()
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters(), lr=0.01)
```

```

# Training
for _ in range(1000):
    name = names[_ % len(names)]
    x = torch.zeros(len(name)-1, 1, n)
    y = torch.tensor([chars.index(c) for c in name[1:]])

    for i, c in enumerate(name[:-1]):
        x[i, 0, chars.index(c)] = 1

    out = model(x)
    loss = loss_fn(out.squeeze(1), y)

    opt.zero_grad()
    loss.backward()
    opt.step()

# Generate
c = "a"
result = c

for _ in range(5):
    x = torch.zeros(1, 1, n)
    x[0, 0, chars.index(c)] = 1
    out = model(x)
    c = chars[out.argmax().item()]
    result += c

print("Generated name:", result)

```

Program 5: ex5.py

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

reviews = ["good movie", "great film", "excellent movie",
           "bad movie", "worst film", "poor movie"]
y = [1, 1, 1, 0, 0, 0]

t = Tokenizer()
t.fit_on_texts(reviews)

X = pad_sequences(t.texts_to_sequences(reviews), maxlen=2)

model = Sequential([
    Embedding(20, 8),
    Bidirectional(LSTM(8)),
    Dense(1, activation="sigmoid")
])

model.compile("adam", "binary_crossentropy", metrics=["accuracy"])
model.fit(tf.constant(X), tf.constant(y), epochs=100, verbose=0)

# Test
test = pad_sequences(t.texts_to_sequences(["good film"]), maxlen=2)
p = model.predict(tf.constant(test), verbose=0)[0][0]

print("Prediction:", "Positive" if p > 0.5 else "Negative")
```

Program 6: ex6.py

```
import nltk
nltk.download('brown')
nltk.download('universal_tagset')
import numpy as np
import nltk
from nltk.corpus import brown
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# Real annotated corpus
data = brown.tagged_sents(tagset='universal')[:1000]

words = sorted(set(w.lower() for s in data for w, t in s))
tags = sorted(set(t for s in data for w, t in s))

wi = {w:i+1 for i,w in enumerate(words)}
ti = {t:i for i,t in enumerate(tags)}

X = [[wi[w.lower()] for w,t in s] for s in data]
Y = [[ti[t] for w,t in s] for s in data]

X = np.array([x[:10] for x in X if len(x) >= 10])
Y = np.array([y[:10] for y in Y if len(y) >= 10])

model = Sequential([
    Embedding(len(words)+1, 16),
    LSTM(32, return_sequences=True),
    Dense(len(tags), activation='softmax')
])

model.compile("adam", "sparse_categorical_crossentropy")
model.fit(X, Y, epochs=3, verbose=0)

p = model.predict(X[:1], verbose=0)
print("Words :", [data[0][i][0] for i in range(10)])
print("Tags  :", [tags[i] for i in np.argmax(p[0], axis=1)])
```

Program 7: ex7.py

```
import numpy as np
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Embedding, LSTM, Dense

# English → French
eng = ["hello", "good morning", "thank you", "good night"]
fra = ["bonjour", "bonjour", "merci", "bonne nuit"]

# Word dictionaries
E = {"hello":1, "good":2, "morning":3, "thank":4, "you":5, "night":6}
F = {"bonjour":1, "merci":2, "bonne":3, "nuit":4}

X = np.array([[1,0], [2,3], [4,5], [2,6]])
Y = np.array([[1,0], [1,0], [2,0], [3,4]])

# Encoder
inp = Input((2,))
e = Embedding(10,8)(inp)
_,h,c = LSTM(16, return_state=True)(e)

# Decoder
dec = Input((2,))
d = Embedding(10,8)(dec)
d = LSTM(16, return_sequences=True)(d,[h,c])
out = Dense(10, activation="softmax")(d)

model = Model([inp,dec],out)
model.compile("adam","sparse_categorical_crossentropy")
model.fit([X,Y],Y[...,:None],epochs=200,verbose=0)

# Test "hello"
p = model.predict([X[:1],Y[:1]],verbose=0)
word = {v:k for k,v in F.items()}[np.argmax(p[0,0])]

print("English: hello")
print("French:",word)
```

Program 8: ex8.py

```
"""
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"]="3"
os.environ["TF_ENABLE_ONEDNN_OPTS"]="0"

import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Flatten, Reshape, Input

# Simple training images
X=np.array([
[[0,1,1,0],[1,0,0,1],[1,0,0,1],[0,1,1,0]],
[[0,1,0,0],[1,1,0,0],[0,1,0,0],[1,1,1,1]],
[[1,1,1,0],[0,0,1,0],[0,1,0,0],[1,1,1,1]]
]*30, dtype="float32")

# Generator
G=Sequential([
    Input(shape=(10,)),
    Dense(32,activation="relu"),
    Dense(16,activation="sigmoid"),
    Reshape((4,4))
])

# Discriminator
D=Sequential([
    Input(shape=(4,4)),
    Flatten(),
    Dense(16,activation="relu"),
    Dense(1,activation="sigmoid")
])

loss=tf.keras.losses.BinaryCrossentropy()
go=tf.keras.optimizers.Adam(0.001)
do=tf.keras.optimizers.Adam(0.001)

# Train GAN
for i in range(1000):
    z=tf.random.normal((8,10))

    with tf.GradientTape() as dt,tf.GradientTape() as gt:
        fake=G(z,training=True)
        real=D(X[np.random.randint(len(X),size=8)],training=True)
        f=D(fake,training=True)

        dl=loss(tf.ones_like(real),real)+loss(tf.zeros_like(f),f)
        gl=loss(tf.ones_like(f),f)

    do.apply_gradients(zip(dt.gradient(dl,D.trainable_weights),
                           D.trainable_weights))
    go.apply_gradients(zip(gt.gradient(gl,G.trainable_weights),
                           G.trainable_weights))

# Generate new image
img=G(tf.random.normal((1,10)),training=False)[0]

plt.imshow(img,cmap="gray")
plt.title("Generated Image")
plt.axis("off")
plt.show()

print("GAN training completed")
print("New synthetic image generated")"""

import os
os.environ["TF_CPP_MIN_LOG_LEVEL"]="3"

import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Flatten, Reshape, Input

(X,_),_=mnist.load_data()
X=X[:10000].astype("float32")/255

G=Sequential([
```

```

        Input((100,)),Dense(256,activation="relu"),
        Dense(784,activation="sigmoid"),Reshape((28,28))
    ])

D=Sequential([
    Input((28,28)),Flatten(),
    Dense(256,activation="relu"),
    Dense(1,activation="sigmoid")
])

loss=tf.keras.losses.BinaryCrossentropy()
go=tf.keras.optimizers.Adam(0.0002)
do=tf.keras.optimizers.Adam(0.0002)

for i in range(5000):
    z=tf.random.normal((32,100))

    with tf.GradientTape() as dt,tf.GradientTape() as gt:
        fake=G(z)
        real=D(X[np.random.randint(len(X),size=32)])
        f=D(fake)

        dl=loss(tf.ones_like(real),real)+loss(tf.zeros_like(f),f)
        gl=loss(tf.ones_like(f),f)

    do.apply_gradients(zip(dt.gradient(dl,D.trainable_weights),
                           D.trainable_weights))
    go.apply_gradients(zip(gt.gradient(gl,G.trainable_weights),
                           G.trainable_weights))

img=G(tf.random.normal((1,100)))[0]

plt.imshow(img,cmap="gray")
plt.title("Generated Digit")
plt.axis("off")
plt.show()

print("GAN training completed")
print("New synthetic image generated")

```